

PrimeWave™ Design Environment Expression Language Reference Manual

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	5
Related Products, Publications, and Trademarks	5
Conventions	6
Customer Support	6
Statement on Inclusivity and Diversity	7

1. Parameter Expression Language (PEL)	8
Overview of PEL	8
Supported Functions in PEL Expressions	9
Parameter/connectDef functions	9
The inst() Function	9
The parent() Function	10
The lineage() Function	12
Operators	13
Math Functions	13
Engineering Suffixes	15
User-Defined PEL Functions	16
Creating and Registering User-Defined PEL Functions	17
Evaluating User-Defined PEL Functions	17
User-Defined PEL Function Example	18
Evaluating PEL Expressions	18
Interpreting Parameter Fields	19
Examples of Parameter Field Interpretation	19
Determining Evaluated Function Values Flow	23
Netlisting PEL Evaluation	24
Conversion to Scientific Notation	25
PEL Undefined Functions	25
Debugging PEL	25

2. Substitution Expression Language (SEL)	27
Overview of SEL	27
SEL Expressions	28

SEL Format Sections	29
SEL @ Operator	29
Multiple Commands in a SEL Expression	30
SEL Data Compatibility	30
3. Simulation Analysis Expression Language	31
Using the Simulation Analysis Expression Language	31
General Syntax	31
Specifying Arguments	32
Adding Quotation Marks for Special Characters	32
Referring to Objects in a Design	33
Accessing Results of Other Expressions	33
Specifying Parameter Conditions	33
Using Logic Values in Expressions	34
Customizing the Simulation Analysis Expression Language	35
Creating Macros	35
Naming Macros	36
Creating Custom Functions	36
Naming Functions and Arguments	37
Function Types	37
Implementing Custom Functions	38
Overriding Existing Functions	39
Structuring the Custom Calculator Function Code	39
A. Mixed PEL/SEL Evaluation	41
Expression Mapping	41

About the PrimeWave Design Environment Expression Language Reference Manual

The *PrimeWave Design Environment Expression Language Reference Manual* describes the Parameter Expression Language (PEL) and the Substitution Expression Language (SEL) used by the PrimeWave Design Environment tool. This manual also describes how to use and customize the Simulation Analysis Expression Language.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeWave Design Environment Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeWave Design Environment tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- Synopsys PrimeWave Design Environment™
- Synopsys Custom Compiler™
- PrimeSim™ Continuum™
- PrimeSim™
- PrimeSim™ HSPICE®

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Parameter Expression Language (PEL)

Describes the Custom Compiler Parameter Expression Language.

Parameter Expression Language (PEL) is an expression format that allows you to use expressions for parameter values.

PEL supports parameter/connectDef functions, operators, mathematical functions, and engineering suffixes.

This section covers the following topics:

- [Overview of PEL](#)
- [Supported Functions in PEL Expressions](#)
- [Evaluating PEL Expressions](#)
- [Debugging PEL](#)

Overview of PEL

PEL allows you to use expressions for parameter values. These can be either string or numeric values.

- **Numeric values:** Use numerical values to perform simple calculations using multiple instance parameters.
- **String values:** Use string values to programmatically set device parameters or to pass string information to custom netlisting procedures.

To help with evaluating PEL expressions, the Custom Compiler tool uses two CDF parameter fields:

- `parseAsPEL`: Allows you to specify whether an expression is a PEL expression.
- `parseAsNumber`: Allows you to specify how to evaluate the PEL expression.

For more information, see [Interpreting Parameter Fields](#)

The Custom Compiler tool also supports evaluation of simple arithmetic operators and basic PEL parameters or `connectDef` functions.

Supported Functions in PEL Expressions

A PEL expression can use parameter/connectDef functions, operators, mathematical functions, engineering suffixes. In addition, PEL expression can also contain Substitution Expression Language (SEL) expressions that use the parameter/connectDef functions `inst()`, `parent()`, and `lineage()` for defining values of various parameters.

PEL supports the following types of functions:

- [Parameter/connectDef functions](#)
- [Operators](#)
- [Math Functions](#)
- [Engineering Suffixes](#)
- [User-Defined PEL Functions](#)

Parameter/connectDef functions

PEL supports three basic parameter/connectDef functions that help you reference parameters, properties, or connection definitions from other instances.

These functions also allow you to specify a default value to be used if no parameter or connection definition is provided. The default value is considered only if the `parseAsNumber` parameter field is set to *nil*.

Note:

Default values do not provide a mechanism for specifying whether the value should be interpreted as either a number or string.

This section covers the following topics:

- [The `inst\(\)` Function](#)
- [The `parent\(\)` Function](#)
- [The `lineage\(\)` Function](#)

The `inst()` Function

The `inst()` function allows you to reference a single property, parameter, or connection definition (`connectDef`) of an instance from another parameter of the same instance. For

example, you can set the width of a parameter to be equal to a scaling of the length with the expression `inst(l)*scale`.

The named parameter must be found on the same instance. The `inst()` function generates an error if the named property, parameter, or `connectDef` does not exist.

You can implement matched pairs using a common `parent()` parameter. For more information, see [The parent\(\) Function](#).

Note:

You cannot use the `inst()` function to track the values found on other instances as matched pairs.

Evaluation of the inst () Function

[Table 1](#) describes the evaluation of the `inst()` function when defining a Custom Compiler parameter and associating it with a PEL function. It also describes the evaluation when any local overrides are present for the parameters.

Table 1 The inst() Function Examples

Custom Compiler Parameter(name = defaultValue)	Associated PEL Function	Local overrides (oaProp/oaParam/oaConnect Def(s))	Evaluated Value
w = 1u	inst(w)	—	1u
w = 1u	inst(foo)	—	Error Generated
w = 1u	inst(w)	w = 2u oaConnectDef: w = vdd	2u
w = 1u	inst(w)	oaConnectDef: w = vdd	vdd
w = 1u	inst(foo)	foo = bar	foo = bar
w = 1u l = 1u ad = inst(w)*inst(l)/2.0	inst(ad)	w = 2u	1e-12
w = 1u l = 1u ad = inst(w)*inst(l)/2.0	inst(ad)	w = 2u oaConnectDef: ad = vdd	vdd

The parent() Function

The `parent()` function allows you to pass parameter and connection definition values through one level of the design hierarchy.

For example, if the database parameter value for `w` on `/I1/I2` is `parent (width)`, then a parameter named `width` must exist on the instance `/I1`.

While evaluating a PEL expression, the PEL expression evaluator considers parameters referenced via the `parent()`.

The `parent()` and function always interpret the default value as a number for evaluation purposes.

Note:

Netlisters in the Custom Compiler environment use the parameters and connection definition values defined in the `parent()` function of an instance to determine the set of parameters to be netlisted for the parent design.

For all the parameters referenced inside the `parent()` function by the `cellView`'s placed instances, the symbol generator supports automatic creation of Parameter Definitions.

Note:

The current instance for the named parameter is not used to avoid circular references. For example, you cannot use the expression `width = parent('width')` as it results in a circular reference.

Evaluation of the `parent()` Function

Table 2 describes the evaluation of the `parent()` function when defining a Custom Design parameter and associating it with a PEL function. It also describes the evaluation when any local overrides are present for the parameters.

Table 2 The `parent()` Function Examples

Parent Parameters/ConnectDefs	Associated CellView Properties	Associated PEL Function	Evaluated At Design Root?	Full Evaluated Value
<code>width = 1u</code>	—	<code>parent(width)</code>	no	<code>1u</code>
<code>width = 1u</code> <code>oaConnectDef: width = vdd</code>	—	<code>parent(width,2u)</code>	no	<code>1u</code>
—	—	<code>parent(width)</code>	no	<code>width</code>
—	<code>width = 2u</code>	<code>parent(width)</code>	Yes	<code>2u</code>
<code>width = 1u (defValue)</code>	<code>width = 2u</code>	<code>parent(width,3u)</code>	Yes	<code>2u</code>
—	—	<code>parent(width,3u)</code>	No	<code>3u</code>
—	—	<code>parent(width,3u)</code>	Yes	<code>3u</code>

Table 2 The parent() Function Examples (Continued)

Parent Parameters/ConnectDefs	Associated CellView Properties	Associated PEL Function	Evaluated At Design Root?	Full Evaluated Value
width width is a parent Custom Compiler parameter, but does not have a defValue field.	—	parent(width,3u)	No	3u
width = ""	—	parent(width,3u)	—	""
oaConnectDef: width = vdd	—	parent(width,3u)	—	"vdd"

The lineage() Function

The `lineage()` function allows you to specify multiple level hierarchical property, parameter, or connection definition (`connectDef`) references. It also tracks the name of the parameter passed for use by the Custom Compiler tool. Tracking the name of the parameter allows you to determine the possible parameter names for all of the parent designs.

Note:

The `lineage()` function does not look in the current instance for the named parameter.

For example, `width = lineage('width')` results in a circular reference and is not allowed.

The Custom Compiler tool performs a search on the parent's parent if it does not find the referenced property, parameter, or `connectDef` in the parent instance. The search continues up the hierarchy until the Custom Compiler tool finds the named parameter or `connectDef`. The `lineage()` function allows you to specify multiple level hierarchical property, parameter, or `connectDef` references.

The `lineage()` function always interpret the default value as a number for evaluation purposes.

The Custom Compiler tool performs a search on the properties or `connectDef` on the top `cellView` if it does not find the referenced parameter or `connectDef` in the root of the design.

Connection Definition Values

Connection definitions are not string type parameters and so cannot have a `parseAsNumber` attribute. The Custom Compiler tool always interprets connection definition values as a string for evaluation purposes.

The following are some important things you should remember about connection definitions:

- The `lineage()` function behaves in the same manner as the `@` operator in SEL if no property or parameter is found. The `lineage()` function uses `connectDef` values.
- When there is no property or parameter value at a level of hierarchy, the connection definition values are searched before proceeding up the hierarchy to search for other properties or parameters.

Operators

Table 3 lists the various operators that are supported in a PEL expression:

Table 3 PEL Supported Operators

Operator	Description
+	Addition
-	Subtraction
/	Division
*	Multiplication
?:	If/Then/Else

Math Functions

The PEL expression evaluator supports math functions when an expression is fully evaluated. An expression is fully evaluated when `parseAsPEL` and `parseAsNumber` are both set to true. Variable references are tracked for all arguments to supported math functions. Table 4 lists the supported math functions:

Table 4 PEL Supported Math Functions

Math Function	Description	Parameter Values	Sample Expression	Evaluated Value
<code>log(x)</code>	Natural logarithm	<code>I=0.1</code>	<code>W=inst(I) * log(100)</code>	0.46052
<code>log10(x)</code>	Base 10 logarithm	<code>I=0.1</code>	<code>W=inst(I) * log10(100)</code>	0.2
<code>exp(x)</code>	e^x	<code>I=0.1</code>	<code>W=inst(I) * exp(1)</code>	0.27183
<code>sqrt(x)</code>	Square root	<code>I=0.1</code>	<code>W=inst(I) * sqrt(4)</code>	0.2

Table 4 PEL Supported Math Functions (Continued)

Math Function	Description	Parameter Values	Sample Expression	Evaluated Value
min(x, y)	Minimum	l=0.1 m=5 nf=4	W=inst(l)*min(inst(nf),inst(m))	0.4
max(x, y)	Maximum	l=0.1 m=5 nf=4	W=inst(l)*max(inst(nf),inst(m))	0.5
abs(x)	Absolute value	l=0.1	W=inst(l) * abs(5.5)	0.55
pow(x, y)	x^y	l=0.1	W=inst(l) * pow(2,2)	0.4
int(x)	Round down to nearest integer.	l=0.1	W=inst(l) * int(2.32)	0.2
floor(x)	Round down to nearest integer.	l=0.1	W=inst(l) * floor(2.59)	0.2
ceil(x)	Round up to the nearest integer.	l=0.1	W=inst(l) * ceil(2.59)	0.3
sin(x)	Sine of x	l=0.4	W=inst(l) * sin(0.5236)	0.2 [#]
cos(x)	Cos of x	l=0.5	W=inst(l) * cos(1.047)	0.25 [#]
tan (x)	Tan of x	l=0.4	W=inst(l) * tan(0.7854)	0.4 [#]
asin(x)	Inverse Sine of x	l=1	W=inst(l) * asin(0.5)	0.5236 [#]
acos(x)	Inverse Cosine of x	l=1	W=inst(l) * acos(0.5)	1.047 [#]
atan(x)	Inverse Tangent of x	l=1	W=inst(l) * atan(1)	0.7854 [#]
atan2(x, y)	Inverse Tangent of x/y	l=1	W=inst(l) * atan2(1,1)	0.7854 [#]
hypot(x, y)	$\sqrt{x^2+y^2}$	l=1	W=inst(l) * hypot(3,4)	5 [#]
sinh(x)	Hyperbolic Sine of x	l=1	W=inst(l) * sinh(0.2)	0.20134 [#]
cosh(x)	Hyperbolic Cosine of x	l=1	W=inst(l) * cosh(0.2)	1.0201 [#]
tanh(x)	Hyperbolic tangent of x	l=1	W=inst(l) * tanh(0.2)	0.19738 [#]
asinh(x)	Inverse hyperbolic sine of x	l=1	W=inst(l) * asinh(0.20134)	0.2 [#]

Table 4 *PEL Supported Math Functions (Continued)*

Math Function	Description	Parameter Values	Sample Expression	Evaluated Value
acosh(x)	Inverse hyperbolic cosine of x	I=1	W=inst(I) * acosh(1.0201)	0.2 [#]
atanh(x)	Inverse hyperbolic tangent of x	I=1	W=inst(I) * atanh(0.19738)	0.2 [#]

indicates that the value is in radians.

Engineering Suffixes

PEL expressions allow you to specify property and parameter values using case sensitive engineering suffixes. These engineering suffixes apply only to the parameter interface.

[Table 5](#) lists the engineering suffixes supported by PEL.

Table 5 *PEL Supported Engineering Suffixes*

Engineering Suffix	Abbreviation	Value
Yotta	Y	10^{24}
Zetta	Z	10^{21}
Exa	E	10^{18}
Peta	P	10^{15}
Tera	T	10^{12}
Giga	G	10^9
Mega	M, meg, X, Meg, MEG	10^6
Kilo	k, K	10^3
Centi	c	10^{-2}

Table 5 *PEL Supported Engineering Suffixes (Continued)*

Engineering Suffix	Abbreviation	Value
Milli	m	10^{-3}
Micro	u	10^{-6}
Nano	n	10^{-9}
Pico	p	10^{-12}
Femto	f	10^{-15}
Atto	a	10^{-18}
Zepto	z	10^{-21}
Yocto	y	10^{-24}

Engineering suffixes are not valid in Tcl built-in mathematical expression evaluator. The Custom Compiler tool provides utility functions for evaluating expressions containing embedded engineering suffixes.

User-Defined PEL Functions

The Custom Compiler tool allows you to create user-defined PEL functions. User-defined PEL functions are implemented in Tcl and are registered at tool startup or within a library startup file. You must register the user-defined function to avoid namespace clashes between Tcl and PEL. User-defined PEL functions use the following Tcl signature:

```
proc tcl_func_name {inst args}
```

PEL user-defined functions provide improved error messages when verifying whether referenced PEL functions are defined within the current session.

Note:

You cannot specify whether the result of a user-defined PEL function is interpreted as a string or a number.

This section covers the following topics:

- [Creating and Registering User-Defined PEL Functions](#)
- [Evaluating User-Defined PEL Functions](#)
- [User-Defined PEL Function Example](#)

Creating and Registering User-Defined PEL Functions

To create a user defined PEL function:

1. Create a Tcl procedure with the proc `tcl_func_name {inst args}` signature.
2. Register the Tcl procedure using the `db::registerParamEvalFunction` command in the Custom Compiler console window.

To register a PEL user-defined function, specify a PEL function name and information about the arguments that function supports by using the `db::registerParamEvalFunction` command in the Custom Compiler console window. The argument information must include:

- The number of arguments the function requires.
- Flags specifying whether the arguments should be evaluated by the PEL parser.

For example, the `parent()` function takes two arguments, neither of which are evaluated.

For information on `db::registerParamEvalFunction` command, refer to the *Custom Compiler Tcl Command Reference Manual*.

Evaluating User-Defined PEL Functions

The following method is used to evaluate a user-defined PEL function:

- The passed PEL arguments in the user-defined function are converted to the corresponding Tcl call signature.
- The registered Tcl function is then evaluated.

The Custom Compiler tool passes the current instance as the first argument to the user-defined function. The second argument contains all of the arguments passed in the evaluated PEL string.

For example, if you call the user-defined PEL function as `foo(one,two,three)` then the `args` variable will be the Tcl list `{one two three}`.

Results of user-defined PEL functions are interpreted as follows for further PEL evaluation:

- As numbers if they are convertible to a real or integer value.
- As strings for further PEL evaluation.

An error message is displayed if any Tcl errors are thrown by the user-defined PEL functions. The Custom Compiler tool allows you to debug the user-defined PEL function.

User-Defined PEL Function Example

The following example shows how to implement a simple Fibonacci generator within a PEL expression.

Note:

The PEL namespace is not a requirement for registering the user defined function.

```
# Tcl implementation of the first 45 elements of the Fibonacci series.
proc PEL::fibonacci {inst args} {
  # declare that this function will return a number
  set i [lindex $args 0]
  if { 1 != [llength $args] || [catch {expr $i%1}] || $i < 1
    || $i > 45} {
    error "expected one integer argument i such that i > 0
    && i < 46"
  }
  uplevel set $typeVar "number"
  set f {1 1 2 3 5 8 13 21 34}
  if { $i < [llength $f] } {
    return [lindex $f $i]
  }
  set j [lindex $f {end-1}]
  set k [lindex $f end]
  set i [expr $i-[llength $f]+1]
  while { $i } {
    incr i -1
    set tmp [expr $j+$k]
    set j $k
    set k $tmp
  }
  return $k
}
```

Evaluating PEL Expressions

This section contains information about how the PEL expression evaluator evaluates the different components of a PEL expression.

This section contains the following topics:

- [Interpreting Parameter Fields](#)
- [Netlisting PEL Evaluation](#)

- [Conversion to Scientific Notation](#)
- [PEL Undefined Functions](#)

Interpreting Parameter Fields

The Custom Compiler tool provides two parameter fields that allow you to specify how the parameter should be evaluated and interpreted.

- `parseAsPEL`: Allows you to specify whether instance parameters should use the PEL expression evaluator to determine the evaluated value of the parameter. The `parseAsPEL` parameter takes boolean values `t` and `nil`.
- `t` specifies that the parameter should be parsed using the PEL expression evaluator
- `nil` specifies that the parameter should not be parsed.
- `parseAsNumber`: Allows you to specify whether a parameter is a number and can be used in PEL expressions. If `parseAsNumber` is set to *false*, then using the parameter in another expression results in a partial evaluation.

Note:

Parameters of type `string` are only considered for PEL evaluation.

Examples of Parameter Field Interpretation

[Table 6](#) describes how PEL evaluation is done depending on different combinations of the `parseAsPEL` and `parseAsNumber` parameter fields:

Table 6 The `parseAsPEL` and `parseAsNumber` Settings

<code>parseAsPEL</code> Setting	<code>parseAsNumber</code> Setting	Result
<code>nil</code>	<code>nil</code>	No evaluation is performed. The value of the parameter is treated as a literal string.
<code>t</code>	<code>nil</code>	PEL evaluation is performed on the parameter. However, the final value is not interpreted as a number. If you use this final value within other numerical PEL expressions, then a partial evaluation is performed.
<code>nil</code>	<code>t</code>	The value of the parameter is interpreted as a number for all subsequent PEL expressions.
<code>t</code>	<code>t</code>	PEL evaluation is performed, and you can use the result of the evaluation in other numerical PEL expressions.

The following examples list evaluation of w (width) and l (length) parameters with different `parseAsPEL` and `parseAsNumber` settings.

- Both Values Parsed As Number
- One Parameter Parsed As Number (partially evaluated)
- Area Set to be Evaluated As PEL (no evaluation)
- Evaluated Area Not Parsed As Number (partially evaluated)
- Parameter Assigned a Design Variable (not fully evaluated)

Both Values Parsed As Number

`parseAsPEL` for both parameters = nil, `parseAsNumber` for both parameters = t

Parameter	Assigned Value	<code>parseAsPEL</code>	<code>parseAsNumber</code>	Evaluated Value
W	1u	nil	t	1u
L	1u	nil	t	1u
Area	<code>inst(w)*inst(l)</code>	t	t	$1e^{-12}$

One Parameter Parsed As Number (partially evaluated)

`parseAsPEL` for both parameters = nil, `parseAsNumber` for one of the parameters = t

Parameter	Assigned Value	<code>parseAsPEL</code>	<code>parseAsNumber</code>	Evaluated Value
W	1u	nil	nil	1u
L	1u	nil	t	1u
Area	<code>inst(w)*inst(l)</code>	t	t	$1u*1u$

Note:

Setting the `parseAsNumber` parameter for W to nil results in a partial evaluation of the area parameter. The area cannot be fully computed because the result of `inst(w)` is not interpreted as a number due to the `parseAsNumber` setting for W.

Area Set to be Evaluated As PEL (no evaluation)

The following table lists evaluation of W(width) and L(length) parameters with different parseAsPEL and parseAsNumber settings.

Parameter	Assigned Value	parseAsPEL	parseAsNumber	Evaluated Value
W	1u	nil	nil	1u
L	1u	nil	t	1u
Area	inst(w)*inst(l)	t	t	1u*1u

Note:

Setting `parseAsPEL` to `t` for `area` results in no evaluation.

Evaluated Area Not Parsed As Number (partially evaluated)

The following table lists evaluation of w (width) and l (length) parameters with different parseAsPEL and parseAsNumber settings.

Parameter	Assigned Value	parseAsPEL	parseAsNumber	Evaluated Value
W	1u	nil	t	1u
L	1u	nil	t	1u
Area	inst(w)*inst(l)	t	nil	1e ¹²
area sqr	inst(area) * inst(area)	t	t	1e ¹² * 1e ¹²

Note:

Turning off the `parseAsNumber` setting for the `area` parameter does not keep the area from being fully evaluated. However, the result is treated as a string. As such PEL expressions using `area` cannot be fully evaluated.

Parameter Assigned a Design Variable (not fully evaluated)

The following table lists evaluation of W and L parameters with different parseAsPEL and parseAsNumber settings.

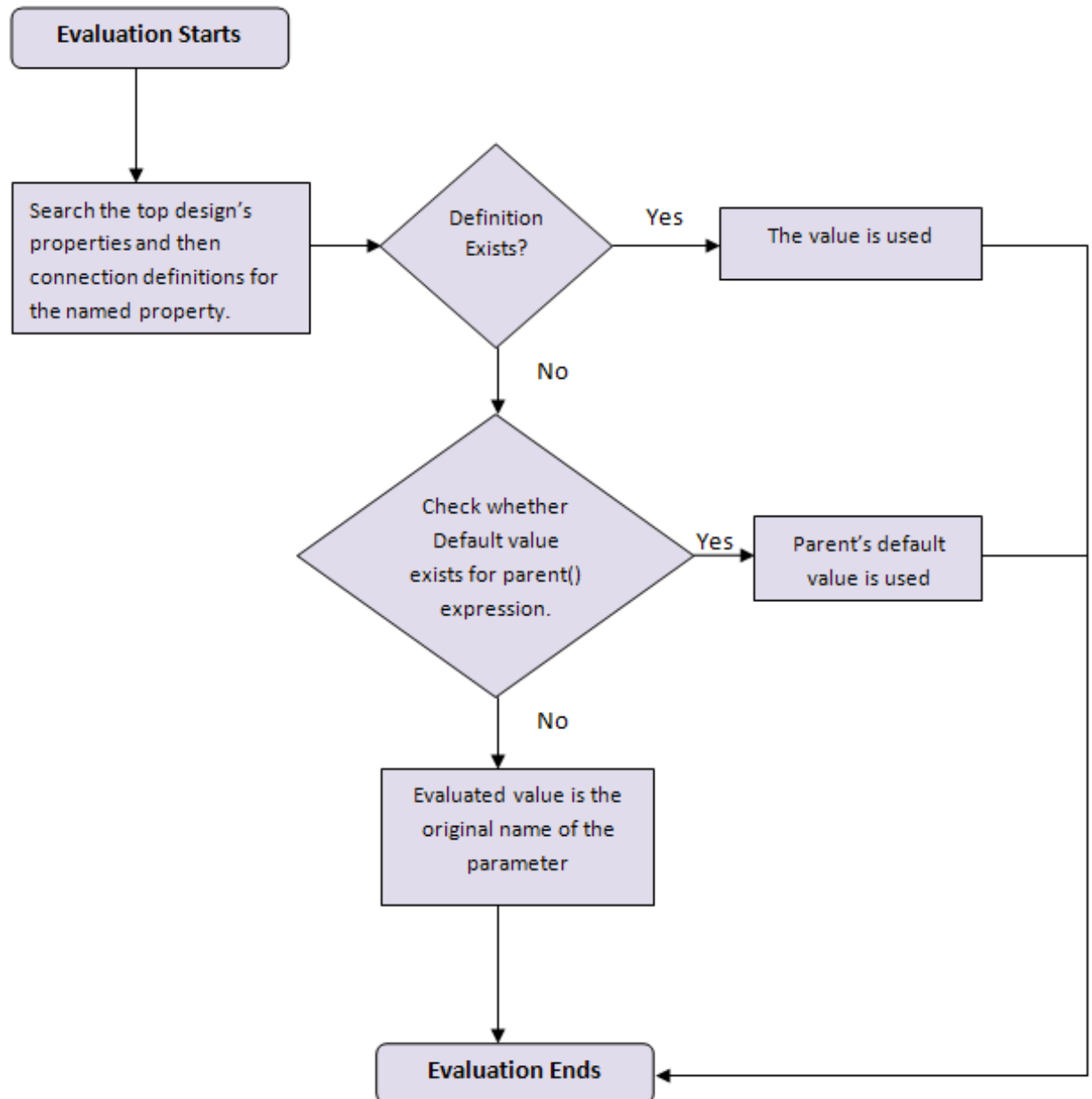
Parameter	Assigned Value	parseAsPEL	parseAsNumber	Evaluated Value
A	1u	t	t	1u
B	SCALE / (inst(A) + 1u)	t	t	SCALE / 2u

Note:

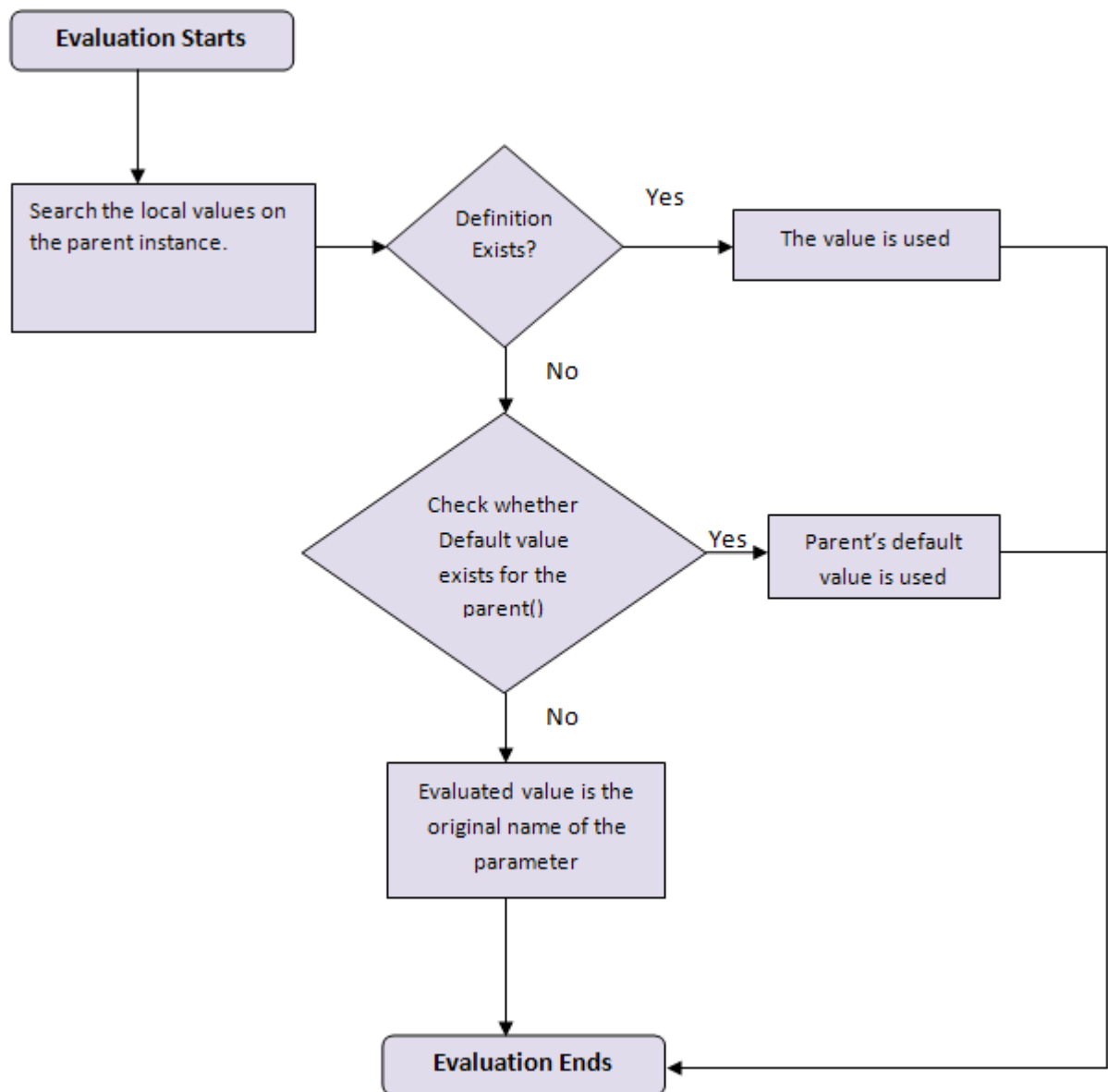
Due to parseAsNumber settings or design variables it may not be possible to fully evaluate the above expression. In such cases the Custom Compiler tool will perform the maximum possible evaluation.

Determining Evaluated Function Values Flow

- The precedence for determining the evaluated value of the `parent()` and `lineage()` functions when the parameter is located in a top design is as follows:



- The precedence for determining the evaluated value of the `parent()` and `lineage()` functions when the parameter is located in a local variable is as follows:



Netlisting PEL Evaluation

When netlisting is performed, the PEL expression evaluator evaluates the `inst()`, `parent()`, and `lineage()` functions. The Custom Compiler tool uses the netlisting value to support the passed parameters.

Function	Netlisting Value
<code>inst()</code>	Computed by copying the value from the referenced parameter without performing any evaluation.
<code>parent()</code>	Name of the parameter.
<code>lineage()</code>	Name of the parameter.

Note:

If a `cellView` property sharing the same name exists in the top of the design, and the evaluation is at the top of the design hierarchy, the netlisting value of the `parent()` and `lineage()` functions are the value of the `cellView` property.

After you netlist, the target simulators perform evaluation of arithmetic expressions and all PEL functions except the `inst()`, `parent()`, and `lineage()` functions.

Conversion to Scientific Notation

The Custom Compiler tool allows you to convert suffixes to equivalent exponents for parameters when `parseAsNumber` is set to *true*. The conversion retains maximum information by substituting the equivalent exponent for the engineering suffix without changing the entered value. For example, if the input string is `1000u`, the converted value will be `1000e-6` instead of `1.0e-3`.

The Custom Compiler environment provides utility functions for converting engineering notation to scientific notation and vice versa.

PEL Undefined Functions

The Custom Compiler tool performs no evaluation, if you define a function name which does not match the built-in functions or any of the user-defined functions that are registered. This allows the target simulators to do the final evaluation.

Debugging PEL

The Custom Compiler tool allows you to debug PEL expressions. The PEL expression evaluator provides textual descriptions of the PEL expression errors with the following information:

- Descriptions of loop detection and parameter evaluation.
- All errors encountered and whether the parameter was referenced.

You can debug a PEL expression by providing a command option to the parameter evaluator. For example, there are three parameters A, B, and C with the values `inst(b)`, `inst(c)`, and `inst(a)` respectively. The value for A contains a circular reference and the following error is returned:

```
Could not resolve circular reference: Parameter "a" references parameter  
"b." Parameter "b" references parameter "c." Parameter "c" references  
parameter "a."
```

2

Substitution Expression Language (SEL)

Describes the Substitution Expression Language (SEL).

Substitution Expression Language (SEL) is a simple substitution format. SEL is used for interpreted labels and property evaluation.

SEL provides text substitution for parameter values and various design information such as `cellViewname` and instance name.

This chapter covers the following topics:

- [Overview of SEL](#)
- [SEL Expressions](#)

Overview of SEL

Substitution Expression Language (SEL) is a substitution language. The following are some of the uses of SEL:

- PrimeSim HSPICE netlisting
- Supporting interpreted labels
- Text substitution for parameter values and various design information, such as `cellView` name and instance name

SEL expressions can appear anywhere within the text. This includes multiple SEL expressions in a line of text and also nested SEL expressions. A SEL expression behaves like a parameter which has the `parseAsNumber` set to false.

SEL does not support evaluation of mathematical expressions. As such, the result of a SEL expression is always a string. SEL evaluation is performed for interpreted labels and property evaluation if any one of the following conditions are satisfied:

- If there is an `oaAppProp` (application-specific property), whose type is `"NLPEXpr"`
- If there is a property of type `oaStringProp` whose value begins with an opening square bracket ("`[`") and which is not an instance override for a parameter.
- If there is a label of App Type `"NLPEXpr"`

SEL Expressions

A SEL expression uses square brackets ("`[`" and "`]`") to bracket substitution expressions. Each expression substitutes its evaluation result back into the original text string between the expressions matching the text inside the open and close brackets.

The format of a SEL expression is as follows:

```
expr := '[' operator key < ':' format < ':' format> > ']'
```

where

- `operator` := (`'+'` | `'@'` | `'~'` | `'|'` | `'.'`)
- `key` := (`paramName` | `"cellName"` | ...)

Table 7 Operators Supported in SEL Expressions

Operator	Description
<code>+</code>	Equivalent to <code>parent()</code> PEL expression
<code>.</code>	Equivalent to <code>inst()</code> PEL expression
<code>@</code>	Equivalent to <code>lineage()</code> PEL expression. Note that this operator is overloaded for label display. Table 8 lists the possible keys for label displays.
<code>~</code>	Same as <code>'.'</code>
<code> </code>	Gets the net connected to the named terminal instance.

The following sections cover:

- [SEL Format Sections](#)
- [SEL @ Operator](#)

- [Multiple Commands in a SEL Expression](#)
- [SEL Data Compatibility](#)

SEL Format Sections

A format section provides the structure for a SEL expression. A SEL expression can have multiple format sections:

- The first format section of a SEL expression is the pattern that is used if SEL evaluation succeeds. If no format section is provided, then the format defaults to %.
- The optional second format section is the pattern to be used if the operation does not succeed, as in the case where a referenced parent parameter does not exist. If a % char is listed in a format, then it is substituted with the result of the operation. If the format needs to include a literal percent sign, then a backslash ("\") is used to escape it.

For example, consider the following SEL expression:

```
[+foo:\%bar == %:bar does not exist]
```

When evaluating this SEL expression the parent parameter `foo` is searched for and the following actions are performed:

- If the parameter exists, then the substituted text is `%bar == value_of_foo`.
- If the parent parameter does not exist, then the substituted text is `bar does not exist`.

SEL @ Operator

The "@" operator in SEL finds both parameters and various design data. The Custom Compiler tool supports the "@" operator in the following labels:

- `cellName`: Returns the name of the cell of the associated object. In the case of an instance this is the cell of the placed master.
- `instanceName`: Returns the name of the associated instance object.
- `partName`: Returns the property on the placed master `cellView` named `partName`. If the property does not exist, then the name of the placed master cell is returned instead.

If the name does not match one of the above keywords, then it is interpreted as a parameter name and the value is found using the same logic as used in the `lineage()` function in PEL.

Multiple Commands in a SEL Expression

SEL language supports multiple SEL expressions for a parameter. SEL also allows you to use nested expression substitution. [Table 8](#) lists the supported multiple SEL expression formats.

Table 8 Multiple SEL Expression Formats

Expression	Description
text[@foo] text [@bar:bar=%] more text	Successive NLP expressions that are not nested.
[@foo:%:[@bar]]	Simple default from another NLP expression.
[@w:w=%;[@l:l=%]]	Appending an NLP expression to the end of a successful format expression.
[@x:x=%:[@aaa:wdef=%]]	Complex default from another NLP expression.
[@vdd:%:[@vss:%:VSS!]]	Complex default from another NLP expression which itself has its own default format.

SEL Data Compatibility

SEL provides support for a single evaluator, which is used by all the other Custom Compiler subsystems that use SEL evaluation. The following are some important points you must remember regarding SEL data compatibility:

- SEL supports all the operators listed in [Table 7](#).
- SEL does not support the `instNamePrefix` property stored at the `cellView` level. For example, `instNamePrefix = [@cellName:%_]` is not supported.

3

Simulation Analysis Expression Language

Describes the Simulation Analysis Expression Language.

This chapter contains the following major sections:

- [Using the Simulation Analysis Expression Language](#)
- [Customizing the Simulation Analysis Expression Language](#)

Using the Simulation Analysis Expression Language

The Simulation Analysis Expression Language is a C-style math expression language, which can perform measurements on simulation results in the PrimeWave Design Environment Results Analyzer and Outputs. The Simulation Analysis Expression Language includes a set of built-in calculator functions for frequently used measurements and can be extended by creating custom calculator functions.

This section contains information the following topics:

- [General Syntax](#)
- [Specifying Arguments](#)
- [Adding Quotation Marks for Special Characters](#)
- [Referring to Objects in a Design](#)
- [Accessing Results of Other Expressions](#)
- [Specifying Parameter Conditions](#)
- [Using Logic Values in Expressions](#)

General Syntax

The following general syntax is used to call functions:

```
func_name([arg_name1=]<arg_value1>, [arg_name2=]<arg_value2>, ...)
```

Strings surrounded by brackets ([]) are optional, and strings surrounded by angle brackets (<>) denote argument values.

Note:

The Simulation Analysis Expression Language is case sensitive.

Specifying Arguments

You can specify arguments by name or position. For example, when you specify a function called `func`, which has the `arg1` and `arg2` arguments, the following invocations are equivalent:

```
func(val1, val2)
func(val1, arg2=val2)
func(arg2=val2, arg1=val1)
```

Note:

Since positional arguments must always precede named arguments, the following example generates an error:

```
func(arg1=val1, val2)
```

Adding Quotation Marks for Special Characters

Generally, you do not need to surround argument values with quotation marks, but you might need to use quotation marks for special characters in value strings. For these cases, you can use either single-quotes (') or double quotes (").

For example, all of the following invocations are equivalent:

```
func(val1, val2)
func(val1, "val2")
func('val1', val2)
```

You must insert quotation marks around argument values that contain:

- commas (,)
- unmatched closed parentheses ())
- single or double quotation marks (' or ")

Note:

Use the opposite quote character for single or double quotation marks.

You do not have to insert quotation marks as long as they adhere to the following rules:

- The argument value contains only alpha-numeric characters or underscores (A-Z, a-z, 0-9, or _).
- The argument does not start with a number.

For example, the following argument must be quoted:

```
func("1net55")
```

This argument does not require quotation marks:

```
func(net55)
```

Referring to Objects in a Design

Functions can provide access to waveform data using object names from the design. The hierarchy delimiter is the forward slash (/) character.

Strings inside functions referring to objects within the design can start with a forward slash character. For example:

```
v(out) is equivalent to v(/out)
i(/I1/M1/D) is equivalent to i(I1/M1/D)
```

Accessing Results of Other Expressions

Generally, results from other expressions can be referenced using the o() function. Output results within the same scope (run directory) can refer to each other directly by name, omitting the o() function. For example, if outputs out1 and out2 are defined in the same scope:

```
out1: func(val1, val2)
out2: o(out1) + func(val3, val4)
```

You can also write the expression for out2 as:

```
out2: out1 + func(val3, val4)
```

Specifying Parameter Conditions

For some operations, you can limit the operation on a waveform to a subset of traces based on the sweep, corner, or Monte Carlo iteration values of the member traces.

The following syntax rules apply when specifying a parametric reduction:

- A single string is used to denote all conditions.
- The syntax for specifying conditions on parameter pName1 is pName1=value1[,value2,...,valueN]. For example, the following condition statements are valid:

```
"Corners=c1,c2,c5"  
"temp=22.5"
```

- For parameters with integer values, you can specify a range of values with the <start:stop[:step]> syntax. For example:

```
"iteration=1, 5,<50:100>,<200:2000:10>"
```

- Conditions on different parameters are delimited by a colon. For example:

```
"A=1:B=2.5,4.6:C=<1:1000:100>"
```

- Not all waveforms parameters must appear in a condition statement. Parameters that do not appear in the parameter conditions statement are not reduced.

When there are multiple values possible for a parameter, the resulting reduced waveform retains that parameter information. If you specify only one value for a parameter, however, the resulting operation discards knowledge of that parameter.

For example, if you reduce a signal that has the A, B, and C parameters with the A=1:B=2,3 condition, the resulting waveform that had parameters B and C, but not parameter A.

Using Logic Values in Expressions

You can use logic values with logic symbolic operators and functions that support logic values as inputs. Logic values and logic signals are different from ordinary numeric values because they are represented internally as strings of bits. Logic values can be defined in binary, octal, decimal, or hexadecimal form.

The following syntax is used for logic values:

```
<nbits>'<radix><value>
```

The following table describes each component of the logic value string:

Component Name	Description
nbits	The bit size of a value (maximum of 32 bits).
radix	Specify h, d, o, or b for hexadecimal, decimal, octal, or binary, respectively.

Component Name	Description
value	The actual logic value.

Note:

You can assign each digit in logic values with a "don't care" or a "tri-state" value using the X and Z characters, respectively.

Customizing the Simulation Analysis Expression Language

You can use the Simulation Analysis Expression Language to create your own functions, which are based on the standard functions included in SAE.

This section contains information on the following topics:

- [Creating Macros](#)
- [Creating Custom Functions](#)
- [Naming Functions and Arguments](#)
- [Function Types](#)
- [Overriding Existing Functions](#)

Creating Macros

Macros are short hand representations of more complex expressions, which can contain expressions and inputs. The core expression of a macro is treated as a plain text string. Before the expression is evaluated, all of the macros are expanded, and their inputs are substituted. New macros can override existing macros, but only if the `-override true` option is specified when you create a macro.

For example, the following macro adds the two inputs together:

```
wf::createMacro f -inputs [list x y] -expression "x + y"  
wf::expr "f(1,3)"
```

This macro calculates the real value of a signal:

```
wf::createMacro vr -inputs [list x] -expression "real(v(x))"  
wf::expr "vr(out)"
```

Naming Macros

The following naming rules apply when naming macros:

- Only alpha-numeric characters or underscores (A-Z, a-z, 0-9, or _) are allowed.
- Names cannot begin with a number.
- Names must be shorter than 100 characters.

Creating Custom Functions

As part of the PrimeWave Design Environment expression framework, you can implement and register new functions, which are then available in the Calculator and can be used in expressions. These functions are implemented and registered in Tcl.

For an example of the concepts discussed in this chapter, see SolvNetPlus article #000039562, [“How to Create a Custom Calculator Function in the PrimeWave Design Environment Using the Tcl Command wf::createFunction.”](#)

The following commands are used to register custom functions:

- `wf::createFunction`
- `wf::createFunctionArgument`

Note:

For a custom function to appear in the Calculator interface, you must source the `expr_lang.tcl` script containing the function. If you make a change to the script, you must close any WaveView session that is running before sourcing the script again.

This section contains information on the following topics:

- [Naming Functions and Arguments](#)
- [Function Types](#)
- [Implementing Custom Functions](#)
- [Overriding Existing Functions](#)
- [Structuring the Custom Calculator Function Code](#)

Naming Functions and Arguments

The following naming rules apply when naming functions and arguments:

- Only alpha-numeric characters or underscores (A-Za-z0-9_) are allowed.
- Names cannot begin with a number.
- Names must be shorter than 100 characters.

Function Types

The following types of functions are used in the simulation analysis expression language:

- [Scalar Measure Type](#)
- [Waveform Conversion Type](#)

All custom functions must be registered as one of these types. The type of a function indicates how it operates on its inputs and gives the expression evaluator advanced knowledge of the end-result of the expression. Function types also allow for much simpler function implementation, since the infrastructure can do the work of handling the different input types (signal/scalar/family of waveforms), and the implementer only has to implement the core functionality.

Scalar Measure Type

Scalar measure functions reduce the dimension of the input arguments by one. Therefore, if the input of a Scalar Measure Type is a signal, the output is a scalar value. If the input is a family of waveforms, the output is a signal. Scalar Measure Type functions accept scalars, signals, and families as input arguments. These functions require that all of any multi-signal inputs have the same number of traces.

The following function calculates the maximum y-value(s) of the input signal or family

```
ymax (v (2) )
```

The Scalar Measure functions are called multiple times when the input is a family of waveforms—once for each trace in the family.

Waveform Conversion Type

Waveform conversion functions return a result of the same dimensionality as the main input.

For example, the following function calculates the Fast Fourier Transform(s) of the input signal or family:

```
fft (v (out) , 1024, 0, 20n)
```

Waveform Conversion functions are called multiple times when the input is a family of waveforms—once for each trace in the family.

Implementing Custom Functions

The Tcl code for all custom functions must be located in the `wf::functions::func_name` namespace, where `func_name` is the name of the function.

Custom functions are implemented using the `wf::createFunction` Tcl command with the argument `-isACE true` and the `wf::createFunctionArgument` command. Custom functions are registered in the PrimeWave Design Environment Calculator.

When `wf::createFunction -isACE true`:

- The value of the `wf::createFunction -functionType` option can be either `waveformConversion` or `scalarMeasure`, according to the return type.
- The `wf::createFunctionArgument` Tcl command with `-dataTypes {signal}` `-argumentType string` is treated as signal, the same as other built-in functions. It is not necessary to specify `resultType`, `analysisName`, or `resultDir` information.
- PrimeWave registers the function and remember the Tcl file it is in.
- When invoking the WaveView tool, PrimeWave finds ACE-based functions and their Tcl files and sends a command to WaveView to source the Tcl files into WaveView.
- When evaluating an expression in PrimeWave, name mapping is handled automatically. The expressions are sent to WaveView to evaluate.

The following example shows a sample implementation and registration for a custom function:

```
puts [info script]
namespace eval wf::functions {

namespace eval merge {

proc execute {args} {
    array set opt $args
    set signal1 $opt(signal1)
    set signal2 $opt(signal2)
    set order $opt(order)
    set offset $opt(offset)
    if {$offset == {}} {
        set offset 0
    }
    if {$order == "signal2 first"} {
        set newSig [sx_equation "concat($signal2,$signal1)+$offset"]
    } else {
        set newSig [sx_equation "concat($signal1,$signal2)+$offset"]
    }
    return $newSig
}
}
}

set signal1 [wf::createFunctionArgument signal1 \
    -description "input signal 1" \
```

```
-label "Signal1" \  
-dataTypes {signal analog} \  
-argumentType string ]  
  
set signal2 [wf::createFunctionArgument signal2 \  
-description "input signal 2" \  
-label "Signal2" \  
-dataTypes {signal analog} \  
-argumentType string ]  
  
set order [wf::createFunctionArgument order \  
-description "signal 1 first, or signal 2 first" \  
-argumentType enum \  
-dataTypes {scalar} \  
-validValues [list signal1_first signal2_first] \  
-defaultValue signal1_first \  
-optional true]  
  
set offset [wf::createFunctionArgument offset \  
-description "real type offset" \  
-argumentType real \  
-optional true]  
  
wf::createFunction merge \  
-description "merge 2 signals into one" \  
-arguments [list $signal1 $signal2 $offset $order] \  
-functionType waveformConversion \  
-isACE true \  
-override true
```

Overriding Existing Functions

You can override any existing function (including built-in functions) in the simulation analysis expression language. If a function is created with the same name as another existing function and the override flag is set to true, the new function replaces the existing one. Once a function is redefined, its initial implementation cannot be restored without restarting the Custom Compiler system.

Structuring the Custom Calculator Function Code

PrimeWave Design Environment output expressions are evaluated by the PrimeWave Design Environment tool and pw_shell, so the code must be available in both applications. To do this, choose one of the following approaches:

Single User

Write the Calculator function code in a single file, and source it from both .custom_compiler.tcl and .pw_shell.tcl. For example:

- calculator_function.tcl: Calculator function implementation and registration
- .custom_compiler.tcl: source /path/to/calculator_function.tcl
- .pw_shell.tcl: source /path/to/calculator_function.tcl

Multi User

Define a customization of the PrimeWave Design Environment startup Tcl code using one of the scope customizations (custom_site, custom_project, custom_user or custom_local). See the *Custom Compiler Environment User Guide*, "Sessions and Settings" chapter of the documentation for additional information. For example:

- primewave/calculator_function.tcl: Calculator function implementation and registration
- primewave/startup.tcl: `db::init primewave/calculator_function.tcl`

A

Mixed PEL/SEL Evaluation

Describes how to use the single evaluator for evaluating PEL or SEL expressions depending on the object type, using expression mapping.

The Custom Compiler tool allows you to combine PEL and SEL expressions. The PEL functions `inst()`, `parent()`, and `lineage()` can reference property values which can contain SEL expressions. Likewise the SEL evaluator can reference parameters.

The Custom Compiler tool implements a single evaluator for evaluation of PEL or SEL expressions depending on the object type.

This chapter describes:

- [Expression Mapping](#)

Expression Mapping

The Custom Compiler tool provides a callback framework for mapping PEL expressions into specific expressions for other target simulators. The Custom Compiler tool uses a `dbExpressionMap` object to describe the mapping.

Caution:

If the target simulator does not support a given expression then use Tcl to report the error.

[Table 9](#) lists the callbacks that are supported by the Custom Compiler tool to implement a new expression mapping:

Table 9 Callbacks Supported by Custom Compiler

Callback	Signature	Description
<code>valueProc</code>	<code><string> valueProc {value}</code>	Called for each real value found in an expression.

Table 9 *Callbacks Supported by Custom Compiler (Continued)*

Callback	Signature	Description
unaryOperatorProc	<string> unaryOperatorProc {operator arg}	Called for each unary operator encountered. The value in <code>arg</code> will have already been mapped by the <code>valueProc</code> if necessary.
binaryOperatorProc	<string> binaryOperatorProc {arg1 operator arg2}	Called for each binary operator encountered. The values in <code>arg1</code> and <code>arg2</code> will have already been mapped by the <code>valueProc</code> if necessary.
ternaryOperatorProc	<string> ternaryOperatorProc {arg1 operator1 arg2 operator2 arg3}	Called for each ternary operator encountered. The values in <code>arg1</code> , <code>arg2</code> , and <code>arg3</code> will have already been mapped by the <code>valueProc</code> if necessary.
functionProc	<string> functionProc {functionName args}	Called to map function call names. All passed-in arguments will have already been mapped by the <code>valueProc</code> if necessary.
designVarProc	<string> designVarProc {desVarName}	Called for every encountered design variable.

Note:

If you do not find a callback in [Table 9](#), then you need not perform any mapping.

The following example illustrates the usage of the expression mapping to map PEL expressions to the PrimeSim HSPICE simulator.

```
proc hspiceBinaryOperator {a oper b} {
  # PrimeSim HSPICE does not support **
  switch -- $oper {
    "**" {
      return "pow([lindex $args 0],[lindex $args 1])"
    }
  }
  return "$a $oper $b"
}
proc hspiceTernaryOperator {a oper1 b oper2 c} {
  error "HSPICE does not support ternary operators."
}
proc hspiceUnaryOperator {oper a} {
  return "$oper $a"
}
```

Appendix A: Mixed PEL/SEL Evaluation Expression Mapping

```
proc hspiceValue {value} {  
  # HSPICE does not support "M". Must be "meg"  
  if { [regsub {M$} $value "meg" result] } {  
    return $result  
  }  
  return $value  
}  
set m [db::createExpressionMap]  
db::setAttr unaryOperatorProc -of $m -value hspiceUnaryOperator  
db::setAttr binaryOperatorProc -of $m -value hspiceBinaryOperator  
db::setAttr ternaryOperatorProc -of $m -value  
hspiceTernaryOperator  
db::setAttr valueProc -of $m -value hspiceValue
```